

ANIMATION

Bewegung am Computer

- Streng technisch betrachtet eine Illusion
- Schneller Wechsel von Bildern mit leichten Änderung
- Wichtig: Frequenz (Bilder pro Sekunde)
- Kann vom Gehirn auch falsch interpretiert werden
 - z.B. Drehrichtung von Rädern mit Speichen

Was bewegt sich?

- Wir haben Ecken, Kanten und Flächen
- Bewegen sich Ecken unterschiedlich, so verformt sich das Objekt (*morphing*)
 - Spezialeffekte: Aufpusten, Schmelzen, ...
- Bewegen sich alle Ecken eines Objektes identisch, so bleibt seine Form erhalten und die Lage ändert sich (*transform*)
 - Häufigere Animationsform

Kontinuierliche Bewegung

- Transformation mit zeitlich variierendem Parameter durchführen
 - Translation um $(x,0,0)$, x pro Sekunde $+1$
 - Rotation mit steigendem Winkel
- Wie in den Übungen
- Sehr einfach umzusetzen
- (Zu) aufwändig für komplexe Bewegungsabläufe

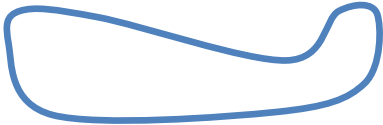
Hierarchische Bewegung

- Erinnerung Planetenmodell
- Sonne liegt in der Galaxie
- Sonne hat ein lokales Koordinatensystem (LKS)
- Erde hat ein LKS
- LKS der Erde hat Position, Orientierung, Skalierung innerhalb des Sonnen-LKSs
- Mond hat LKS innerhalb des Erd-LKSs

Koordinatensysteme animieren

- Nicht das Objekt „Erde“ um die Sonne rotieren
- ...sondern das LKS der Erde rotieren
- Alles innerhalb des LKS wird ebenfalls rotiert, z.B. der Mond
- Durch komplexe LKS-Hierarchien können komplizierte Bewegungen durch mehrere einfache abgebildet werden
- Mond um Sonne? Mond um Erde um Sonne!

Translation

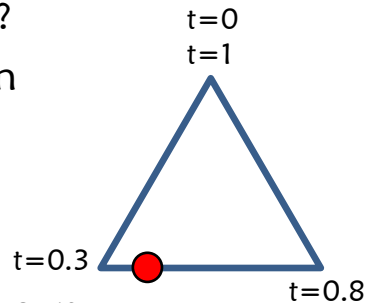
- Translation in eine Richtung sehr einfach
 - Koordinate pro Sekunde +1
- Translation hin und her schon besser
 - Sinus eines Winkels, Winkel pro Sekunde +1
- Translation entlang eines beliebigen Pfades
 - Pfad als Funktion
 - Pfad als Polygon

Pfadanimation

- Pfad als Funktion
 - Ein Parameter t („wie weit bin ich“)
 - Einfach auszurechnen
 - Beliebig genau
- Beispiel
 - $f(t) = (\cos(t), \sin(t), t)$
 - Position des Objekts zum Zeitpunkt 0: (x, y, z)
 - Position zum Zeitpunkt t : $(x, y, z) + f(t)$

Pfad als Polygon

- Parameter t muss „umgerechnet“ werden
 - Welche Strecke des Polygons?
 - Wie weit auf dieser Strecke?
- Aufwändiger auszurechnen
- Polygon „eckiger“



- Beispiel
 - Position zum Zeitpunkt $t = 0.4$?

Mehr als Translation...

- Blickrichtung
 - Aktuelle Position p zum Zeitpunkt t ausrechnen
 - Position q zum Zeitpunkt $t + 0.0...1$ ausrechnen
 - Objekt durch Rotation in Richtung des Vektors $q-p$ ausrichten
- Bewegung der Beine beim Laufen
 - Bei Pixelgrafik einfach Bildfolge „abspielen“
 - Wie geht das bei Vektorgrafik?

Gelenksystem

- Starre Objekte werden durch Gelenke verbunden
- Jedes Gelenk erlaubt eine eingeschränkte Bewegung
 - Winkelbereich um eine Achse
 - Drehungen um verschiedene Achsen mit verschiedenen Winkelbereichen
 - Kugelgelenk
- Orientierung ergibt sich aus Gelenkwinkeln
- Beispiel: Arme, Beine

Skelett

- Bewegung wird nur auf Knochen definiert
 - Gelenke sind Ecken
 - Knochen sind Kanten zwischen zwei Ecken
- Zu jedem Knochen gehört ein 3D-Objekt
 - Wird später „um“ den Knochen gelegt
- Lage und Rotation des 3D-Objekts bestimmt sich aus den angrenzenden Gelenken
 - Bei Bewegung ständig neu berechnen

Keyframes

- Design legt bestimmte Posen eines Skeletts fest
 - z.B. jetzt Arm oben, in 2s Arm unten
- Zwischen zwei Posen wird interpoliert
 - Typischerweise lineare Interpolation
- Damit ist zu jedem Zeitpunkt eine Pose definiert, keine „Sprünge“
- Wer prüft Einhaltung der Gelenkbeschränkungen?

Kinematik

- Bewegungslehre der Mechanik
- Gelenksysteme heißen **kinematische Kette**
- **Direkte Kinematik** = Berechnung der Position der starren Objekte anhand der einzelnen Gelenkwinkel
 - Wie bei lokalen Koordinatensystemen

Inverse Kinematik - Idee

- Umkehrung der direkten Kinematik
- Statt alle Winkel in den Gelenken zu kennen, ist nur die Lage des Endobjekts bekannt
- Beispiel: Finger drückt einen Knopf
 - Aufgabe: Gelenkwinkel für Finger, Hand, Ellenbogen, Schulter bestimmen
 - Winkel müssen Gelenkbedingungen erfüllen

Inverse Kinematik

- Wird häufig auf eine Kette von Knochen angewendet, die jeweils mit Gelenken verbunden sind
- Abzweigungen/T-Stücke/Kreuzung führen zu deutlichen Komplikationen
- Der Anfang der Kette ist häufig fest
 - Z.B. der Arm im Schultergelenk
- Ende kann frei bewegt werden und soll Ziel erreichen

System von Ungleichungen

- Position von Schulter und Finger fest
- Variablen: Position von Ellenbogen und Hand
- Bedingungen
 - Abstände Schulter zu Ellenbogen und Ellenbogen zu Hand gegeben
 - Alle Gelenkwinkel müssen im erlaubten Wertebereich liegen

Sinnvolle Lösung finden

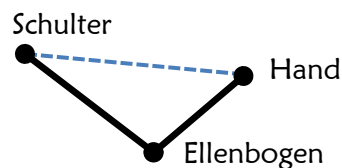
- System hat normalerweise viele Lösungen
 - Kann auch keine haben
- Welche ist am besten?
 - Realistisch – Wie erkennt der Computer das?
 - Schnell zu erreichen
 - Wenig „eingewickelt“
- Möglichkeit: Für jedes Gelenk „natürliche Richtung“ definieren
 - Summierte Abweichungen minimieren

Lösungsansätze

- Kein direktes, schnelles und universelles Verfahren bekannt
 - Algebraische Lösung
 - Matrizen invertieren, mathematisch aufwändig
 - Numerische Lösung
 - Optimierungsverfahren findet lokales Optimum, benötigt guten Startwert
 - Geometrische Lösung (z.B. bei *two bone IK*)
 - Winkel aus Geometrie bestimmen, z.B. Cosinussatz
 - Einfache geometrische Näherungen

Direkte Lösung

- Für typische Arm/Bein-Situationen
 - Zwei Knochen mit einem Gelenk dazwischen
- Möglichkeit für unser Beispiel:
 - Fingerposition = Handposition
 - Dreieck mit drei bekannten Seitenlängen
 - Winkel berechnen
 - Ein Winkel in Schulter noch frei



Geometrische Näherungslösung

- Gibt zwei sehr bekannte Algorithmen
- Beide sind einfach zu implementieren
- Liefern keine direkte Lösung, sondern schrittweise Verbesserung
- Müssen häufig mehrfach durchlaufen werden für gute Lösung
- Können auch scheitern und keine Lösung liefern oder Gelenkbeschränkungen verletzen

CCD-Algorithmus

- Cyclic Coordinate Descent
- Beginne mit letztem Knochen/letztem Gelenk der Kette
- Setze den Winkel im Gelenk so, dass der Knochen mit dem Zielpunkt auf einer Geraden liegt
- Wandere nun die Kette nach oben zum Anfang und wiederhole für jedes Gelenk den obigen Schritt
- Wiederhole das ganze Verfahren mehrfach
- Beispiel an der Tafel

FABRIK-Algorithmus

- Forward And Backward Reaching IK
- Rückwärts: Setze Endknoten auf Ziel
- Ziehe vorletzten Knoten auf richtigen Abstand zu letztem
- Analog für alle weiteren Knoten bis zum Start
- Vorwärts: Setze ersten Knoten auf Ursprungsposition
- Ziehe nächsten Knoten auf richtigen Abstand... etc. für alle
- Wiederhole das ganze Verfahren

CCD vs. FABRIK

- CCD verändert Winkel
 - Dadurch direkte Kontrolle über Gelenkbeschränkungen
 - Viele Wiederholungen bei langen Ketten nötig
 - Ergebnisse teilweise unrealistisch
- FABRIK verschiebt Punkte
 - Aufwändiger auf Gelenkbedingungen zu achten
 - Konvergiert schneller / weniger Wiederholungen
 - Durch den Richtungswechsel entstehen ausgewogenere/natürlichere Ergebnisse
 - Kann besser mit mehreren Zielen umgehen

Noch komplexere Animation

- In 3D-Systemen/Game Engines häufig auf „einfacher“ Physik basierend
 - Gravitation/Beschleunigte Bewegung, Reibung, Elastizität
- Es existieren auch sehr komplizierte Modelle
 - Soft body physics, Federmodelle
- Verlässt schnell den „einfachen“ Bereich der Computergrafik und wir hiermit auch das Kapitel...